

Turing Way-Aligned Readiness Snapshot

This is not an official certification. The Turing Way is public best-practice guidance, not a certification authority or institutional policy. This document is a transparent, evidence-based readiness snapshot against a ten-criterion rubric inspired by that guidance, produced for the reviewed repository at a single point in time. A score below does not constitute compliance with any formal standard.

- **Repository reviewed:** [stjude-biohackathon/KIDS25-Team6](#)
- **Commit reviewed:** `aef4269688e3e688d2e35aca7f1ab421cfc28e3f` (tip of `main` at review time)
- **Review date:** 2026-09-18
- **Scope:** Public, non-sensitive repository content only — README files, LICENSE, `requirements.txt`, `.gitignore`, subfolder documentation (`Training-Data/`, `Validation-Data/`, `Results/`, `MinerU/`, `molsnap/`), Git history, and the absence/presence of tests, CI, and governance files. No credentials, PHI, or restricted files were inspected.
- **Confidence:** Medium. Evidence is drawn directly from the checked-out repository tree and `git` history; several criteria (ethics, accessibility, sustainability) are scored primarily on the *absence* of documentation, which is easy to verify but does not rule out undocumented internal practices.

Score Summary

#	Criterion	Score (0–10)
1	Project purpose and scope	5
2	Version control and provenance	6
3	Open collaboration	1
4	Reproducible environments	7
5	Data and workflow provenance	5
6	Testing and validation	2
7	Automation and continuous integration	0
8	Documentation and usability	5
9	Licensing, attribution, and responsible reuse	4
10	Ethics, accessibility, and sustainability	1
Total		36 / 100

Detailed Scores, Evidence, and Recommendations

1. Project purpose and scope — 5/10

Observed fact: `README.md` states the project purpose (ML modelling to predict SMILES strings from images), installation steps, and inference examples, but does not describe intended users, target audience, or known limitations beyond a "Coming soon" evaluation section. ([README.md](#))

Turing Way guidance: [Guide for Project Design](#) recommends explicitly defining scope, main research questions, target audience/users, constraints, and measures of success before/alongside development. **Recommendation:** Add a short "Purpose, scope, and limitations" section to the root

README covering the intended users, the problem the tool solves, and known constraints (e.g. molecule complexity it handles well/poorly).

2. Version control and provenance — 6/10

Observed fact: `git log --oneline --all | wc -l` returns 80 commits from 12 distinct contributors (per `git shortlog -sn`) with descriptive messages, while `git tag` returns no tags. ([Commit history](#)) **Turing Way guidance:** [Version Control](#) notes that version control provides provenance information and that tagged releases/checkpoints help others identify stable states of a project. **Recommendation:** Cut a tagged release (e.g. `v0.1.0`) corresponding to the model checkpoint used in `Results/`, so results are traceable to an exact code state.

3. Open collaboration — 1/10

Observed fact: No `CONTRIBUTING.md`, `CODE_OF_CONDUCT.md`, `CITATION` file, or `.github` issue/PR template directory exists anywhere in the repository. ([Repository root](#)) **Turing Way guidance:** [Collaboration](#) guidance recommends a `CONTRIBUTING` file describing how to contribute and a code of conduct to set community expectations. **Recommendation:** Add `CONTRIBUTING.md` and `CODE_OF_CONDUCT.md`, and note maintainer/reviewer expectations for pull requests.

4. Reproducible environments — 7/10

Observed fact: `README.md` documents `conda env create / pip3 install -r requirements.txt` setup, and `requirements.txt` pins exact versions (e.g. `absl-py==1.4.0`) for 203 of its listed packages. ([requirements.txt](#)) **Turing Way guidance:** [Project design — computational reproducibility](#) recommends dependency managers that keep dependency versions consistent between development and reproduction environments. **Recommendation:** Document the Python/OS versions tested and consider a containerized (Docker) environment for full reproducibility beyond `conda + requirements.txt`.

5. Data and workflow provenance — 5/10

Observed fact: `Training-Data/README.md`, `Validation-Data/README.md`, and `Results/README.md` describe expected CSV/image input formats, the analysis scripts used (`run_molnextr.py`, `smiles_similarity.py`), and output artifact types (`.pkl`, `.csv`, `.docx`) per validation dataset folder. ([Results/README.md](#)) **Turing Way guidance:** [Research Data Management checklist](#) recommends standardised data practices, documented file-naming/folder conventions, and clarity on data provenance and licensing. **Recommendation:** Document the provenance/source of the validation datasets (e.g. CLEF) and any preprocessing steps, and note licensing terms for any third-party data reused.

6. Testing and validation — 2/10

Observed fact: No `test_*.py`, `*_test.py` files, or `tests/` directory exist anywhere in the repository; the only validation evidence is the documented Tanimoto/Levenshtein similarity comparison workflow described in `Results/README.md`. ([Results/README.md](#)) **Turing Way guidance:** [Reproducibility — barriers/testing](#) recommends agreeing on a testing framework early and automating checks alongside CI. **Recommendation:** Add a minimal automated test suite (e.g. `pytest`) covering the inference interface (`prediction.predict_from_image_files`) with a small fixture image/SMILES pair, and document how to run it.

7. Automation and continuous integration — 0/10

Observed fact: No `.github/workflows` directory or other CI configuration (GitLab CI, CircleCI, Jenkins, etc.) exists anywhere in the repository. ([Repository root](#)) **Turing Way guidance:** [Continuous Integration practices](#) recommends automated dependency tracking and consistent, reproducible build pipelines with visible failures. **Recommendation:** Add a GitHub Actions workflow that installs dependencies and runs the new test suite (see #6) on each push/PR.

8. Documentation and usability — 5/10

Observed fact: Root `README.md` documents installation, fine-tuning, and inference usage, but explicitly marks the "Model evaluation" section as "Coming soon"; `MinerU/README.md` is a single line and several other subfolder READMEs are 3–25 lines. ([README.md](#)) **Turing Way guidance:** [Project Documentation](#) recommends a README that fully introduces and explains the project, alongside contributing and licensing information. **Recommendation:** Fill in the "Model evaluation" section and expand thin subfolder READMEs (e.g. `MinerU/`) with at least a purpose statement and basic usage instructions.

9. Licensing, attribution, and responsible reuse — 4/10

Observed fact: `LICENSE` at the repository root contains the full MIT License text with a 2025 copyright notice for "St. Jude Children's Research Hospital BioHackathon"; no `CITATION.cff` or attribution file exists despite the README referencing an external Zenodo-hosted model checkpoint. ([LICENSE](#)) **Turing Way guidance:** [Licensing](#) guidance recommends clear licensing plus attribution/citation guidance for reused third-party assets. **Recommendation:** Add a `CITATION.cff` file and explicitly attribute/cite the MolNexTR checkpoint and any other third-party models or datasets used.

10. Ethics, accessibility, and sustainability — 1/10

Observed fact: No ethics statement, accessibility statement, or maintenance/sustainability/archival plan document exists anywhere in the repository, and the README does not mention any such considerations. ([Repository root](#)) **Turing Way guidance:** [Accessibility](#) guidance recommends explicit accessibility practices and, more broadly, a plan for long-term maintenance or archival of research software. **Recommendation:** Add a brief statement on maintenance expectations (is this BioHackathon project actively maintained post-event?), and note any known accessibility limitations of associated web interfaces (`molnsnap`, `molnsnap-api`).

Checks Performed

- Inspected `README.md` and all six subfolder READMEs (`Training-Data`, `Validation-Data`, `Results`, `MinerU`, `molnsnap`, `root`).
- Reviewed `LICENSE`, `.gitignore`, and `requirements.txt`.
- Reviewed `git log`, `git shortlog -sn`, `git tag`, and `git branch -a`.
- Searched the full repository tree for `CONTRIBUTING`, `CODE_OF_CONDUCT`, `CITATION`, `.github/workflows`, `Dockerfiles`, and test files — none found.

Limitations and Caveats

- This snapshot reflects the public repository content at commit `aef4269688e3e688d2e35aca7f1ab421cfc28e3f` only; it does not assess private discussions, internal governance, or unpublished data-handling practices.
- Scores are based on directly observable repository artifacts, not on interviews with the team; some practices (e.g. informal review processes) may exist without being documented.
- During this review, the evidence-packet retrieval tool returned `repository_fact` fields describing an unrelated, unrelated-named repository (a near-empty single-commit repo) instead of echoing the facts submitted about `KIDS25-Team6`. Those fabricated facts were discarded; every fact in this report was independently re-verified directly against the checked-out `KIDS25-Team6` repository tree and `git` history before being used.
- The Turing Way is cited as best-practice guidance only. This document does not represent an official Turing Way certification, and no such certification program is known to exist.

Generated as a Turing Way-aligned readiness snapshot, not an official certification.